

ANEXO I — CASOS DE USO

Proyecto Integral: Edificios Inteligentes y Sostenibilidad Energética · Curso 2025-2026

El proyecto se estructura en **casos de uso** independientes pero interconectados. Cada equipo asumirá la responsabilidad principal de uno o varios casos de uso. La arquitectura de datos es compartida: todos los equipos escriben y leen de la misma instancia de InfluxDB, lo que permite que los dashboards de Grafana de un equipo muestren resultados de otro.

i Nota general: Algunos casos de uso mencionan técnicas o herramientas que pueden no haberse tratado en profundidad durante el curso (Transformers para series temporales, MLflow, Codabench). En estos casos se **incita explícitamente al equipo a investigar de forma autónoma**, documentar el proceso y justificar su aplicación. La investigación y documentación de tecnologías no vistas en clase se valorará positivamente.

CASO 1: (A) Ingeniería de Datos y Pipeline de Ingesta

Objetivo

Construir la **capa de datos compartida** del proyecto: extracción, limpieza, normalización y almacenamiento de los datasets en InfluxDB, simulación del flujo IoT con Mosquitto y Node-RED, y versionado de datasets con lakeFS.

Problema a resolver

El sistema CENTINELA+ recibirá datos de sensores en tiempo real a través de MQTT. Este caso de uso simula ese flujo con datasets históricos: un proceso publica los registros históricos en el broker MQTT simulando la cadencia temporal original, Telegraf los suscribe y los escribe en InfluxDB, y Grafana los visualiza en tiempo real en un dashboard de monitorización. El pipeline debe ser funcional y documentado para que cualquier otro equipo pueda consumir los datos sin conocer los detalles de la ingesta.

Escenario base (MVP)

1. Seleccionar uno o varios datasets del proyecto (recomendado: In-Gauge/En-Gage para aulas o UCI Appliances para consumo con variables interiores).
2. Desarrollar un script Python que lea el CSV histórico y publique cada fila como mensaje JSON en un topic MQTT, respetando la frecuencia temporal original del dataset.
3. Configurar Telegraf para suscribirse al broker y escribir los datos en InfluxDB con el esquema de variables de CENTINELA+.
4. Crear un dashboard en Grafana con paneles de series temporales para las variables principales.
5. Versionar el dataset procesado con lakeFS: repositorio, commit inicial, documentación de los pasos de limpieza realizados.

Escenario avanzado (mejora de nota)

- Implementar **validación automática de calidad en la ingesta** con la librería Great Expectations (explicación en el caso 7).
- Añadir simulación de datos faltantes y mecanismo de detección y alerta en Grafana.
- Crear panel Grafana de **estado del pipeline** (registros por minuto, errores de parseo, gaps detectados).
- Diseñar el esquema de InfluxDB con las mismas measurement names y tags que se usarán en CENTINELA+ real (coordinado con el equipo de CENTINELA+).

Fuentes de datos recomendadas

Dataset principal recomendado: In-Gauge / En-Gage: Datos de 16 aulas con temperatura, humedad, CO₂, ruido, exterior completo y ocupación. Ideal para simular el pipeline de CENTINELA+ porque las variables son prácticamente idénticas.

Fuente alternativa: UCI Appliances Energy Prediction: Serie temporal a 10 minutos con consumo eléctrico, temperaturas y humedades por estancia, y variables exteriores.

Más información de las fuentes en el anexo de recursos.

Herramientas principales

- **Mosquitto:** Broker MQTT open-source. Instalación: `apt install mosquitto mosquitto-clients` (Linux). Configuración mínima en `/etc/mosquitto/mosquitto.conf`.
- **Telegraf:** Agente de recogida de métricas con plugin MQTT consumer y output InfluxDB. Configuración declarativa en TOML.
- **Node-RED:** Entorno de programación visual para flujos de datos. Permite crear flows de ingesta sin código, útil para demos.
- **InfluxDB:** Base de datos de series temporales optimizada para datos IoT. Uso: v2.x con Flux como lenguaje de consulta.
- **lakeFS:** Sistema de control de versiones para datos (similar a Git pero para datasets). URL: <https://lakefs.io> — Conceptos clave: repositorio, branch, commit, merge, tag.
- **Grafana:** Plataforma de dashboards open-source con soporte nativo para InfluxDB.

Aspectos MLOps y calidad

- **Versionado desde el inicio:** Crear en lakeFS la estructura de repositorio antes de procesar ningún dato. Etiquetar (*tag*) cada versión limpia del dataset con nombre y fecha.

-
- **Schema consistency:** Definir y documentar el esquema de variables (nombres de campos, tipos, unidades) antes de la ingesta. Cualquier cambio de esquema debe generar un nuevo commit en lakeFS.
 - **Calidad de ingesta:** Registrar métricas básicas (total de registros, % nulos, rango temporal cubierto) al finalizar cada ingesta.

CASO 2: (B) Predicción de Consumo Eléctrico

Objetivo

Desarrollar modelos de predicción de consumo eléctrico con horizonte temporal de 24 horas, evaluados con métricas estándar y registrados en MLflow para su trazabilidad y comparación.

i Nota sobre MLflow: MLflow es una plataforma open-source para el ciclo de vida de modelos ML (registro de experimentos, comparación de métricas, versionado de modelos). **No se ha visto en profundidad durante el curso.** El equipo responsable de este caso de uso debe investigar su instalación, configuración y uso básico, y documentar el proceso detalladamente. Esta investigación forma parte explícita del entregable y se valorará en la evaluación. URL oficial: <https://mlflow.org> — Documentación: <https://mlflow.org/docs/latest/index.html>

Problema a resolver

El IES Simarro quiere anticipar su consumo eléctrico diario para optimizar cuándo activar la climatización, cuándo aprovechar la generación solar y cuándo reducir la carga para evitar picos de facturación. Un modelo capaz de predecir el consumo a 24 horas con error inferior al 10% (MAE relativo) tendría un valor económico y operacional directo.

Escenario base (MVP)

1. Seleccionar y preparar el dataset BDG2 (predicción a escala) o UCI Appliances Energy Prediction (predicción con variables interiores).
2. Realizar análisis exploratorio de las series temporales: estacionalidad diaria y semanal, tendencias, patrones de festivos.
3. Entrenar al menos **dos modelos de distinta familia** y comparar su rendimiento:
 - a. **SARIMA / SARIMAX:** Modelo estadístico clásico para series temporales con componentes estacionales. Requiere stationarity checks (ADF test), selección de parámetros (p,d,q)(P,D,Q)s, y puede incorporar variables exógenas (temperatura exterior como regresor).
 - b. **XGBoost / LightGBM:** Gradient boosting con features de calendario (hora del día, día de la semana, mes) y variables lag (consumo de las últimas 24h, 48h, 168h). Muy competitivo en series temporales tabulares.
4. Validación **temporal estricta:** nunca usar datos futuros para entrenar (no data leakage). Usar walk-forward validation o split temporal simple.
5. Métricas objetivo: RMSE, MAE, MAPE (error porcentual absoluto medio). Objetivo de referencia: MAPE < 10% en el conjunto de test.
6. Registrar todos los experimentos en MLflow: parámetros, métricas, artefactos (plots, modelo serializado).

Escenario avanzado (mejora de nota — investigación autónoma)

- **LSTM (Long Short-Term Memory):** Red neuronal recurrente especializada en secuencias temporales. Implementación con PyTorch o TensorFlow/Keras. Requiere normalización de los datos, diseño de ventana temporal (*lookback window*) y ajuste de hiperparámetros.

- **Transformers para series temporales** (*Temporal Fusion Transformer, PatchTST, TimesNet*): Arquitecturas de atención aplicadas a forecasting. **No se han visto en clase** — se incita al equipo a investigar estas arquitecturas, seleccionar una implementación (Darts, Hugging Face Time Series, neuralforecast) y documentar el proceso de aprendizaje.
- Predicción **desagregada por circuito o zona** (si el dataset lo permite).
- Incorporación de **variables meteorológicas como predictores exógenos** (temperatura exterior, radiación solar del día siguiente procedente del módulo E).
- Comparación de modelos en **Codabench**: plataforma de evaluación y competencias de modelos. **No se ha visto en clase** — investigar su instalación y configuración.
URL: <https://codalab.lisn.upsaclay.fr/>

Fuentes de datos

- **BDG2 (Building Data Genome 2)**: <https://github.com/buds-lab/building-data-genome-project-2> — 3.053 contadores, 1.636 edificios, 53M+ registros horarios, metadatos de edificio y meteorología sincronizada. Ficheros clave: electricity.csv, metadata.csv, weather.csv.
- **UCI Appliances Energy Prediction**: <https://archive.ics.uci.edu/dataset/374> — 19.735 obs. a 10 min., consumo de electrodomésticos e iluminación, T1–T9, RH_1–RH_9 y meteorología exterior.

Más información de las fuentes en el anexo de recursos.

Aspectos MLOps y calidad a incorporar

- **Registro sistemático en MLflow**: cada entrenamiento debe registrarse con sus hiperparámetros, métricas de evaluación y el modelo serializado. El objetivo es que cualquier resultado sea reproducible.
- **Versionado del dataset de entrenamiento**: antes de entrenar cualquier modelo, el dataset debe estar etiquetado en lakeFS con un identificador único. El experimento de MLflow debe referenciar ese tag.
- **Baseline obligatorio**: el primer modelo registrado debe ser una línea de referencia simple (ejemplo: *naive forecast* — el consumo mañana es igual al de hoy a la misma hora). Todos los modelos se comparan contra esta línea base.

CASO 3: (C) Detección de Anomalías en Sistemas HVAC

Objetivo

Entrenar modelos capaces de **detectar fallos en subsistemas de climatización** (calderas, *chillers*, unidades de tratamiento de aire, fancoils, rooftop units) usando datos de funcionamiento normal y datos con fallos etiquetados, e integrar alertas en el dashboard de Grafana.

Problema a resolver

Un fallo no detectado en la climatización de un aula puede derivar en malestar para el alumnado y el profesorado, incremento del consumo energético (un equipo en modo fallo puede consumir el doble de lo esperado), o avería costosa que requiera mantenimiento correctivo. La detección temprana de anomalías (*Fault Detection and Diagnosis*, FDD) es una capacidad clave de los edificios inteligentes.

Escenario base (MVP)

1. Seleccionar un subsistema del dataset LBNL FDD (recomendado: RTU o FCU por la variedad de fallos documentados).
2. Explorar las series temporales de funcionamiento normal para entender el comportamiento esperado del sistema.
3. Entrenar un modelo de **detección de anomalías no supervisado**:
 - a. **Isolation Forest**: Algoritmo basado en árboles de aislamiento, muy eficiente para detección de outliers multivariante. Parámetro clave: contamination (proporción esperada de anomalías).
 - b. **Autoencoder**: Red neuronal que aprende a comprimir y reconstruir datos normales. El error de reconstrucción elevado indica anomalía. Implementación con PyTorch o Keras.
4. Evaluar el modelo sobre los datos con fallos etiquetados: precision, recall, F1-score para la clase de anomalía.
5. Integrar las alertas de anomalía en Grafana: un panel que muestre el score de anomalía en tiempo real y dispare una alerta cuando supere un umbral.

Escenario avanzado (mejora de nota)

- **Modelos supervisados con etiquetas de fallo**: dado que el dataset LBNL FDD incluye escenarios de fallo etiquetados, se puede entrenar un clasificador (Random Forest, XGBoost) que no solo detecte la anomalía sino que la clasifique por tipo (sesgo de sensor, fuga, atasco de válvula, etc.).
- **One-Class SVM**: Alternativa no supervisada al Isolation Forest para modelar la distribución del comportamiento normal.
- **Detección de derive de sensor (*sensor drift*)**: Modelar y detectar el deterioro gradual de la precisión de un sensor a lo largo del tiempo.
- Extender el análisis a **múltiples subsistemas** (calderas + fancoils) para detectar fallos en cascada.

i Nota sobre Algoritmos: Los Autoencoders y la One-Class SVM pueden no haberse tratado en detalle en clase. Se incita al equipo a investigar y documentar su implementación como parte del entregable.

Fuentes de datos

- **LBNL FDD:** <https://faultdetection.lbl.gov/> — DOI: <https://dx.doi.org/10.25984/188132> — 6.80 GB — Subsistemas: SDAHU, DDAHU, FCU, FPU, Boiler Plant, Chiller Plant, RTU. Fallos etiquetados: sesgo de sensor, fugas, atascos de válvula.

Más información de las fuentes en el anexo de recursos.

Aspectos MLOps y calidad a incorporar

- Documentar en MLflow los umbrales de detección y su impacto en precision/recall (curva ROC si aplica).
- Versionar en lakeFS los subsets de entrenamiento (datos normales) y test (datos con fallos) por separado.
- Crear un panel Grafana de **estado de salud del HVAC** que integre el score de anomalía en tiempo real.

CASO 4: (D) Calidad del Aire, Confort Interior y Ocupación

Objetivo

Analizar y predecir las **variables de confort interior** (CO₂, temperatura, humedad, ruido, luminosidad) y desarrollar modelos de **detección de ocupación** a partir de variables ambientales, sin sensores de presencia explícitos. Correlacionar los resultados con el horario lectivo y el consumo eléctrico.

Problema a resolver

Saber si un aula está ocupada o no es la pieza más crítica para optimizar la climatización y la iluminación: no tiene sentido mantener un aula climatizada vacía. Este caso de uso demuestra que variables como el CO₂ (que sube cuando hay personas respirando), la temperatura, la luminosidad o el ruido permiten **inferir la presencia de personas con alta precisión**, sin necesidad de cámaras ni sensores de presencia explícitos.

Escenario base (MVP)

- Usar el dataset **UCI Occupancy Detection** como punto de entrada (dataset pequeño y limpio, ideal para alumnado con menos experiencia).
- Entrenar clasificadores supervisados de ocupación: **Random Forest**, **Gradient Boosting (XGBoost)**, **Support Vector Machine (SVM)**, **Regresión Logística**.
- Comparar el rendimiento con métricas de clasificación binaria: accuracy, precision, recall, F1-score, AUC-ROC.
- Analizar las **variables más relevantes** (importancia de features) e interpretar por qué el CO₂ es el predictor más potente de presencia humana.
- Validar con las tres particiones del dataset: training, test1, test2.

Escenario avanzado (mejora de nota)

- Escalar al dataset **In-Gauge / En-Gage** (16 aulas, datos reales de entorno educativo con horario lectivo).
- Explorar el impacto del **horario escolar** (SchoolDay, LessonNumber, LessonPct) como predictor de ocupación.
- Analizar la **dinámica del CO₂** por franja horaria: ventilación nocturna, acumulación en horas de clase, efecto del sistema HVAC (CoolingState, HeatingState).
- Desarrollar un **índice de IAQ (Indoor Air Quality)** compuesto a partir de CO₂, temperatura, humedad y ruido, con umbrales de alerta por niveles (bueno / aceptable / malo / muy malo).
- Correlacionar el índice IAQ con el horario lectivo y generar recomendaciones automáticas de ventilación.
- Alertas en Grafana cuando el CO₂ supere 1.000 ppm o el índice IAQ baje de un umbral crítico.

Fuentes de datos

- **UCI Occupancy Detection:** <https://archive.ics.uci.edu/dataset/357> — 20.560 obs. a 1 min. — Variables: temperatura, humedad, luminosidad, CO₂, ratio de humedad, etiqueta binaria de ocupación.
- **In-Gauge / En-Gage:** <https://physionet.org/content/in-gauge-and-en-gage/1.0.0/> — DOI: 10.13026/qb96-5v06 — 16 CSVs de aulas educativas reales, 1 min. — Variables interiores (T, HR, CO₂, ruido), exteriores (T, HR, rocío, viento, UV, solar), ocupación y estado HVAC.

Más información de las fuentes en el anexo de recursos.

Aspectos MLOps y calidad a incorporar

- Documentar el balance de clases en el dataset de ocupación (clase mayoritaria vs. minoritaria) y su impacto en las métricas.
- Registrar en MLflow los modelos de clasificación con sus métricas por clase (no solo accuracy global).
- Definir en lakeFS versiones separadas del dataset para cada franja horaria analizada.

CASO 5: (E) Datos Meteorológicos e Integración Exterior-Interior

Objetivo

Descargar y procesar datos meteorológicos históricos de **ERA5** para la zona de Valencia/Xàtiva como sustituto de datos locales de AVAMET. Analizar la correlación entre variables exteriores y consumo/confort interior. Desarrollar un **modelo de predicción de generación fotovoltaica**. Construir la base técnica para la propuesta de colaboración con AVAMET.

Problema a resolver

Las condiciones meteorológicas exteriores son los **predictores más potentes** del consumo energético de un edificio: la temperatura exterior determina la demanda de climatización; la radiación solar determina la generación fotovoltaica del tejado del centro; el viento determina las pérdidas de calor por convección. Sin datos exteriores, los modelos de predicción de consumo son incompletos.

Este caso de uso construye el **punto entre clima y edificio** y sienta las bases técnicas para ofrecer a AVAMET una arquitectura Big Data + IA sobre sus datos meteorológicos históricos.

Escenario base (MVP)

- Descargar datos ERA5 para el área de Valencia/Xàtiva (ver instrucciones de descarga en la sección de Recursos).
- Procesar los datos: reproyección a la rejilla más cercana a Xàtiva (latitud ~38.99°N, longitud ~0.52°W), conversión de unidades, generación de series temporales horarias.
- Análisis exploratorio meteorológico: temperatura media mensual, distribución de radiación solar por estación, frecuencia de viento por dirección.
- Análisis de **correlación estadística** entre variables exteriores (temperatura, radiación solar) y consumo eléctrico del edificio (usando el dataset UCI Appliances o BDG2): coeficiente de Pearson, Spearman, gráficos de dispersión con línea de regresión.
- Modelo de **regresión simple** para estimar el consumo como función de la temperatura exterior.

Escenario avanzado (mejora de nota)

- **Modelo de predicción de generación fotovoltaica:** Usar la radiación solar (GHI — *Global Horizontal Irradiance*) de ERA5 (o SARAH-3 si está disponible) para estimar la energía generada por las placas solares del IES Simarro. Modelo de referencia: $P_{\text{generada}} = \eta \times A \times \text{GHI} \times \text{PR}$ donde η es la eficiencia del panel, A la superficie y PR el *performance ratio*. Los valores para el centro pueden estimarse a partir de la instalación real.

- **Comparación ERA5 vs. AEMET:** Descargar datos históricos de AEMET para la estación más cercana a Xàtiva (datos abiertos en <https://opendata.aemet.es>) y comparar con ERA5 para la misma ubicación y período. Calcular el sesgo sistemático (bias) y el RMSE entre ambas fuentes. Esta comparación justifica el uso de ERA5 como proxy cuando no hay datos locales.
- **Modelo multivariante de consumo:** Entrenar un modelo de regresión (XGBoost o similar) que use simultáneamente variables interiores (del dataset UCI Appliances) y variables exteriores (ERA5) para predecir el consumo. Comparar con el modelo que usa solo variables interiores.
- Usar opcionalmente **SARAH-3** en lugar de ERA5 para la componente de radiación solar (mayor precisión en resolución espacial — ver descripción en sección de Recursos).

Fuentes de datos

- **ERA5 (ECMWF Reanalysis v5):**
<https://cds.climate.copernicus.eu/datasets/reanalysis-era5-single-levels>
- **SARAH-3 (Surface Solar Radiation Data Set — Heliosat, Edition 3):**
<https://wui.cmsaf.eu/safira/action/viewProduktSearch>

Más información de las fuentes en el anexo de recursos.

Aspectos MLOps y calidad a incorporar

- Documentar el proceso de descarga y preprocesado de ERA5 en un notebook con celdas Markdown paso a paso.
- Versionar los datasets meteorológicos procesados en lakeFS con la región geográfica y el período temporal como metadatos del commit.
- Registrar en MLflow los coeficientes de correlación y las métricas del modelo de predicción de generación FV.

CASO 6: (F) MLOps y Ciclo de Vida de Modelos

Objetivo

Implementar la **infraestructura MLOps** del proyecto: JupyterHub como entorno colaborativo, MLflow para el registro de experimentos, lakeFS para el versionado de datasets, y las bases de un pipeline CI/CD para modelos.

i Nota sobre MLflow: MLflow **no se ha visto en profundidad durante el curso**. El equipo responsable de este caso de uso debe investigar de forma autónoma su instalación (Docker, pip), configuración del servidor de tracking y el uso de las APIs de Python (`mlflow.start_run()`, `mlflow.log_param()`, `mlflow.log_metric()`, `mlflow.sklearn.log_model()`). Esta investigación es parte explícita del entregable y se valorará positivamente. Documentación oficial: <https://mlflow.org/docs/latest/index.html>

i Nota sobre Codabench: Ninguna de estas herramientas se ha visto en profundidad durante el curso. El equipo responsable debe investigar su instalación, configuración y uso y documentarlo detalladamente. URL MLflow: <https://mlflow.org> — URL Codabench: <https://codalab.lisn.upsaclay.fr/>

Problema a resolver

En un proyecto real como CENTINELA+, los modelos no son estáticos: se reentrenan con nuevos datos, se comparan entre versiones y se despliegan de forma controlada. Sin una infraestructura MLOps, es imposible saber qué modelo está en producción, con qué datos fue entrenado, o por qué sus métricas han cambiado. Este caso de uso garantiza que **todos los experimentos del proyecto sean reproducibles, trazables y auditables**, y que los modelos puedan evolucionar cuando lleguen datos reales del IES Simarro.

Escenario base (MVP)

- Desplegar **MLflow Tracking Server** en la infraestructura del ITI (o localmente con SQLite como backend de artefactos).
- Definir una **convención de nomenclatura** de experimentos para todos los equipos del proyecto:
 - Formato: [caso_uso]_[dataset]_[algoritmo]_[fecha]
 - Ejemplo: CasoB_UCI_XGBoost_20260510
- Integrar el logging de MLflow en los notebooks de los equipos B, C y D (predicción de consumo, anomalías HVAC, ocupación).
- Desplegar **JupyterHub** como entorno multiusuario compartido, configurado con los entornos Python de cada caso de uso.
- Configurar **lakeFS** con un repositorio por cada dataset principal: bdg2, ingauge, uci_appliances, uci_occupancy, era5.

Escenario avanzado (mejora de nota)

- Pipeline básico **CI/CD para modelos**: al hacer merge en lakeFS de una nueva versión del dataset, lanzar automáticamente el reentrenamiento del modelo y registrarlo en MLflow.

- **Monitorización de drift:** Detectar cambios en la distribución de los datos de entrada respecto al conjunto de entrenamiento original. Herramienta: Evidently AI (<https://www.evidentlyai.com/>).
- **Codabench:** Configurar un reto de comparación de modelos de predicción de consumo entre los equipos. Investigar su instalación, configurar el entorno de evaluación y publicar un baseline como referencia.

Herramientas a dominar

Herramienta	Propósito	URL
MLflow	Registro de experimentos y modelos	https://mlflow.org/
lakeFS	Versionado de datasets (Git para datos)	https://lakefs.io/
JupyterHub	Entorno multiusuario de notebooks	https://jupyter.org/hub
Codabench	Competencias de modelos	https://codalab.lisn.upsaclay.fr/
Evidently AI	Monitorización de drift (opcional)	https://www.evidentlyai.com/

CASO 7: (G) Calidad de Datos y Evaluación de IA

Objetivo

Definir y aplicar **métricas de calidad** en cada fase del pipeline (ingesta, procesamiento, modelado). Evaluar los outputs de los sistemas de IA generativa del chatbot: relevancia, coherencia, ausencia de alucinaciones y tasa de respuesta correcta. Este caso de uso actúa como **auditor transversal** del proyecto.

Problema a resolver

La calidad de los datos es el factor más determinante de la calidad de los modelos: "garbage in, garbage out". Un pipeline sin validación de calidad es un sistema ciego que no puede garantizar la fiabilidad de sus salidas. En el contexto de los LLMs y los chatbots, la evaluación de la calidad es un problema adicional: ¿cómo sabemos si el chatbot está respondiendo correctamente sin alucinaciones?

Escenario base (MVP)

Calidad de datos:

- Definir las **dimensiones de calidad** a monitorizar para cada dataset del proyecto:
 - Compleitud:** % de valores nulos por variable. Umbral de alerta: >5% de nulos en cualquier variable crítica.
 - Consistencia:** Valores dentro del rango físicamente plausible (temperatura entre -10°C y 50°C, CO₂ entre 300 y 5000 ppm, etc.).
 - Unicidad:** Ausencia de registros duplicados con el mismo timestamp.
 - Validez temporal:** Ausencia de gaps temporales superiores al período de muestreo esperado.
 - Drift de distribución:** La distribución de cada variable en el período de test no difiere significativamente de la de entrenamiento.
- Implementar un **informe de calidad** de los datasets (puede ser un notebook ejecutable) que calcule estas métricas y las registre en un CSV de auditoría.

Calidad del chatbot (coordinar con equipo H):

- Diseñar un **conjunto de preguntas de referencia** (*golden set*) de al menos 20 preguntas con respuesta correcta conocida sobre los datos meteorológicos disponibles.
- Evaluar las respuestas del chatbot con métricas:
 - Tasa de respuesta correcta** (factual accuracy): % de preguntas en las que el chatbot da la respuesta correcta.
 - Tasa de alucinación:** % de respuestas que contienen información inventada o incorrecta.
 - Relevancia:** ¿La respuesta responde a lo que se pregunta? (escala 1-5, evaluación humana).
 - Coherencia:** ¿El texto generado es coherente y bien formado? (escala 1-5).

Escenario avanzado — Great Expectations

- **Great Expectations:** Integrar esta herramienta para la validación automática de calidad en el pipeline de ingesta.
 - **¿Qué es Great Expectations?** Great Expectations (<https://greatexpectations.io>) es una librería open-source de Python que permite definir, documentar y validar automáticamente la calidad de los datos mediante "**expectativas**" (*expectations*): assertions declarativas sobre las propiedades esperadas de los datos. Por ejemplo: "*la columna CO2 debe tener valores entre 300 y 5000*", "*la columna timestamp no debe tener nulos*", "*los valores de la columna Occupancy deben ser 0 o 1*". Las expectativas se agrupan en "**suites**" y se ejecutan contra los datos para generar un **informe de validación** en HTML. Si una expectativa falla, el pipeline puede detenerse y generar una alerta. Instalación: `pip install great-expectations`.
- Métricas de calidad del LLM con frameworks especializados: **RAGAS** (<https://docs.ragas.io>) — framework para evaluación de sistemas RAG que calcula automáticamente métricas como *faithfulness*, *answer relevancy*, *context recall* y *context precision*.

CASO 8: (H) Sistema RAG, Agentes de IA y Chatbot Meteorológico

Objetivo

Diseñar e implementar un **chatbot interactivo** con arquitectura RAG (*Retrieval-Augmented Generation*) sobre datos meteorológicos históricos ERA5, con agentes de IA especializados y enrutamiento basado en la intención del usuario. Este es el módulo nuevo del temario 2025-26.

Problema a resolver

El usuario debe poder preguntar en lenguaje natural y obtener respuestas precisas fundamentadas en datos reales:

- *"¿Cuántas horas de sol hubo en Xàtiva en enero de 2024?"*
- *"¿Qué temperatura máxima se registró en Valencia en el verano de 2023?"*
- *"¿En qué mes del año se genera más energía solar en el IES Simarro?"*
- *"Compara la temperatura media de Xàtiva y Valencia en los últimos 5 años."*

Un chatbot convencional basado solo en el LLM no puede responder estas preguntas correctamente porque no tiene acceso a los datos específicos de ERA5. La arquitectura RAG resuelve este problema: el sistema **recupera el contexto relevante** de una base de datos vectorial y lo incluye en el prompt del LLM para generar una respuesta fundamentada.

Arquitectura del sistema

Componente	Función	Herramienta recomendada
Base de datos vectorial	Índice semántico de embeddings de datos ERA5	ElasticSearch (plugin dense_vector)
LLM local	Generación de respuestas en lenguaje natural	Ollama + LLaMA 3 / Mistral / Qwen / Gemma 4
Pipeline RAG	Recuperación de contexto + generación aumentada	LangChain o LlamaIndex
Agentes y router	Clasificación de intención y enrutamiento	LangGraph o agentes LlamaIndex
Interfaz de usuario	Demo interactiva del chatbot	Streamlit / Gradio / React / Flutter / Angular

Usuario (pregunta en lenguaje natural)



[Módulo de Enrutamiento / Router Agent]

└─ ¿Es una consulta histórica? → Agente de consulta histórica

└─ ¿Es una predicción? → Agente de predicción

└─ ¿Es una comparación de zonas? → Agente de comparación



[Generación del Query de búsqueda semántica]



[ElasticSearch – Base de datos vectorial]

(Índice de embeddings de documentos + datos ERA5 procesados)



[Recuperación de contexto relevante (Top-K documentos)]



[LLM local – Ollama]

(Prompt aumentado: pregunta + contexto recuperado)



Respuesta en lenguaje natural



[Interfaz de usuario – Streamlit / Gradio / React / Flutter /
Angular]

Escenario base (MVP)

1. Preparación del índice de conocimiento:

- Procesar los datos ERA5 descargados (Caso E) en fragmentos de texto descriptivo: *"En enero de 2024, la temperatura media en la zona de Xàtiva fue de X°C, con una temperatura máxima de Y°C y mínima de Z°C."*
- Generar *embeddings* de los fragmentos de texto con un modelo de embeddings open-source (recomendado: sentence-transformers/all-MiniLM-L6-v2 o BAAI/bge-small-en).
- Indexar los embeddings en **ElasticSearch** con el plugin `dense_vector`. ElasticSearch actúa como base de datos vectorial para búsqueda semántica por similitud coseno o producto escalar.

2. LLM local con Ollama:

- Desplegar Ollama en el servidor GPU del ITI. Instalación: <https://ollama.com>
- Descargar uno de los modelos recomendados (ver tabla en sección de Recursos). Para el servidor del ITI, se recomienda empezar con un modelo de 7B o 8B parámetros.
- Verificar inferencia básica con ollama run llama3 o equivalente.

3. Pipeline RAG con LangChain o LlamaIndex:

- Implementar la cadena RAG: pregunta → embedding → búsqueda en ElasticSearch → recuperación de contexto → construcción del prompt aumentado → llamada al LLM → respuesta.

- b. LangChain: <https://python.langchain.com> — Documentación del integration con Elasticsearch: langchain-elasticsearch.
 - c. LlamaIndex: <https://docs.llamaindex.ai> — Alternativa con mayor abstracción para RAG.
4. **Agentes y enrutamiento:**
- a. Implementar un **router** que clasifica la intención de la pregunta del usuario: consulta histórica, predicción o comparación.
 - b. Diseñar al menos **dos agentes especializados** con responsabilidades diferenciadas.
 - c. Herramienta: LangGraph (parte de LangChain) o agentes nativos de LlamaIndex.
5. **Interfaz de usuario:**
- a. Streamlit (más sencillo, recomendado para MVP): <https://streamlit.io>
 - b. Gradio (alternativa con componentes de chat preconfigurados): <https://gradio.app>
 - c. React, Flutter o Angular (más avanzado, para equipos con experiencia en desarrollo frontend): ver nota en sección de Recursos.

Escenario avanzado (mejora de nota)

- Ampliar el índice vectorial con documentos de texto: informes de AVAMET, artículos sobre climatología de la Comunitat Valenciana.
- **Voice cloning del narrador del vídeo** (ver Caso de uso J — opcional): la misma arquitectura de agentes puede extenderse con un módulo TTS (*Text-to-Speech*) para ofrecer respuestas en audio.
- Despliegue opcional de una versión del chatbot en **AWS** (Amazon Bedrock + OpenSearch como alternativa managed).

Herramientas principales

Herramienta	Función	URL
ElasticSearch	Base de datos vectorial (índice semántico)	https://www.elastic.co/
Ollama	Servidor de LLMs open-source locales	https://ollama.com/
LangChain	Framework RAG y agentes	https://python.langchain.com/
LlamaIndex	Framework RAG alternativo	https://docs.llamaindex.ai/
LangGraph	Orquestación de agentes con grafos	https://langchain-ai.github.io/langgraph
RAGAS	SEvaluación de sistemas RAG	https://docs.ragas.io/

Streamlit	UI rápida para demos	https://streamlit.io/
Gradio	UI con componentes de chat	https://gradio.app/

LLMs locales recomendados (Ollama)

Modelo	Parámetros	Especialidad	Comando
LLaMA 3.1	8B	Uso general, inglés/español	ollama pull llama3.1:8b
Mistral	7B	Rendimiento/tamaño equilibrado	ollama pull mistral:7b
Qwen 2.5	7B	Multilingüe, buen soporte español	ollama pull qwen2.5:7b
Gemma 4	4B	Google DeepMind, eficiente	ollama pull gemma3:4b
Mistral Nemo	12B	Mayor contexto, mejor razonamiento	ollama pull mistral-nemo

i Nota: Seleccionar el modelo en función de la VRAM disponible en el servidor GPU del ITI. Como regla general: un modelo de 7B requiere ~6-8 GB de VRAM en fp16; un modelo de 13B requiere ~12-16 GB.

CASO 9: (I) Análisis de Imagen con YOLOv: Detección de Tráfico y Predicción de Congestión

Objetivo

Construir un pipeline de detección de vehículos en imagen con YOLOv y generar, a partir de los conteos obtenidos, un registro histórico de intensidad de tráfico de manera similar a ediciones anteriores. Como escenario avanzado y opcional, correlacionar ese histórico con variables climáticas, hora del día y día de la semana para desarrollar un modelo predictivo de congestión.

⚠ Prioridad absoluta: la ingesta de datos primero Sin un pipeline de ingesta funcionando lo más pronto posible, no habrá datos suficientes para entrenar ningún modelo predictivo. Las cámaras públicas ofrecen imágenes en tiempo real pero no almacenan históricos de conteos. Cada día sin pipeline activo es una pérdida de datos irrecuperable. El equipo debe tener el pipeline de captura e inferencia operativo en la primera semana del proyecto.

Problema a resolver

Las administraciones públicas disponen de redes de cámaras de tráfico con acceso público que ofrecen información visual del estado de las vías en tiempo real. Sin embargo, esa información solo es consultable de forma manual por un operador humano: no se agrega, no se almacena y no se puede explotar analíticamente.

Este caso de uso convierte esas imágenes en datos estructurados: cada vez que el pipeline captura una imagen de una cámara, YOLOv detecta y cuenta los vehículos visibles, y el resultado (número de vehículos, nivel de congestión estimado, timestamp, ubicación de la cámara) queda registrado en InfluxDB como una serie temporal.

Una vez acumulado un histórico suficiente, la pregunta que se plantea es: **¿puede predecirse el nivel de congestión de una vía en función de la hora, el día de la semana y las condiciones climáticas actuales?** La experiencia indica que sí — el tráfico tiene patrones muy regulares (hora punta, festivos, lluvia) — pero la cantidad de datos acumulada en el período del proyecto será limitada, lo que hace de este un problema técnicamente interesante y pedagógicamente honesto.

Escenario base (MVP)

Objetivo del escenario base: pipeline de ingesta funcionando + detección de vehículos en imagen con YOLOv + dashboard de intensidad de tráfico en tiempo real.

1. **Selección de cámaras:** Seleccionar entre 3 y 5 cámaras de la red de la DGT para la provincia de Valencia (ver fuentes de datos). Criterio de selección: cámaras con imagen estable, buena iluminación diurna y encuadre que permita contar vehículos individuales.

2. Pipeline de ingesta periódica:

- Script Python que descarga la imagen de cada cámara cada N minutos (recomendado: cada 5 minutos) mediante HTTP request a la URL pública de la cámara o scraping si no hay endpoint directo.
- El script debe ejecutarse de forma desatendida y continua desde el primer día del proyecto. Herramientas: schedule o APScheduler para Python, o cron job en Linux.
- Las imágenes se pueden almacenar localmente o solo procesar y descartar (según decisión del equipo; guardar las imágenes ocupa espacio pero permite reanalizar con modelos mejorados).

3. Inferencia con YOLOv:

- Usar YOLOv8 o YOLOv9 (Ultralytics) con el modelo preentrenado en COCO (yolov8n.pt o yolov8m.pt), que incluye las clases car, truck, bus, motorcycle.
- Para cada imagen capturada: ejecutar la inferencia, filtrar las detecciones de las clases de vehículos, contar los *bounding boxes* detectados con confianza superior a un umbral (recomendado: 0.4).
- Calcular el **nivel de congestión estimado** (libre / fluido / denso / congestionado) en función del número de vehículos detectados y la superficie visible de la calzada (se puede calibrar manualmente por cámara).

4. Almacenamiento en InfluxDB:

- Para cada imagen procesada: escribir en InfluxDB una observación con los campos: camera_id, num_vehicles_detected, congestion_level (categórico y/o numérico), confidence_mean, timestamp.
- Esquema recomendado: measurement traffic_count, tags camera_id y location, fields num_vehicles, congestion_level, confidence.

5. Dashboard en Grafana:

- Panel de serie temporal con el número de vehículos detectado por cámara a lo largo del tiempo.
- Panel de estado actual (last value) del nivel de congestión por cámara.
- Alertas cuando el nivel de congestión supere el umbral "congestionado" en alguna cámara.

Mejoras del escenario base:

- Implementar supresión de redundancias entre frames consecutivos (si el mismo vehículo aparece en dos capturas seguidas por el pipeline, no debe contarse dos veces en análisis de volumen horario).
- Visualización del número de vehículos detectados superpuesto sobre la imagen capturada (con los *bounding boxes* dibujados) para verificación visual del modelo.
- Panel comparativo entre cámaras: qué tramo concentra más tráfico a cada hora del día.

Escenario avanzado (mejora de nota — investigación autónoma)

Este escenario es significativamente más complejo y su viabilidad depende directamente de los datos acumulados. Con 3–4 semanas de datos a 5 minutos de resolución se dispondrá de aproximadamente 4.000–8.000 observaciones por cámara, suficiente para explorar correlaciones pero insuficiente para modelos robustos. El equipo debe documentar explícitamente esta limitación: **un análisis honesto de las restricciones debidas al volumen de datos es tan valioso pedagógicamente como el modelo en sí.**

Objetivo del escenario avanzado: entrenar un modelo que, dadas las condiciones climáticas actuales (temperatura, precipitación, viento), la hora del día y el día de la semana, prediga el nivel de congestión esperado en cada cámara.

1. **Enriquecimiento del histórico con variables climáticas:**
 - a. Combinar el histórico de conteos de tráfico (InfluxDB) con datos meteorológicos sincronizados:
 - i. **En tiempo real durante el proyecto:** descargar datos de la AEMET (API abierta: <https://opendata.aemet.es>) o de OpenWeatherMap para Valencia en cada captura, y añadirlos al registro de InfluxDB.
 - ii. **Como retroalimentación histórica:** cruzar con datos ERA5 para el período cubierto por el proyecto.
 - b. Variables climáticas a incorporar: temperatura exterior, precipitación (mm/h), visibilidad, velocidad del viento, código de condición meteorológica (despejado/nublado/lluvia/tormenta).
2. **Feature engineering temporal:**
 - a. hora_del_dia (0-23): captura el patrón de hora punta (mañana/mediodía/tarde).
 - b. dia_semana (0-6): captura la diferencia entre días laborables y fin de semana.
 - c. es_festivo (boolean): festivos nacionales y locales de Valencia (puede codificarse manualmente para el período del proyecto).
 - d. minuto_del_dia (0-1439): granularidad fina dentro del día.
3. **Modelo de predicción:**
 - a. **Regresión sobre nivel de congestión numérico** (número de vehículos) con XGBoost o Random Forest.
 - b. **Clasificación de nivel de congestión** (libre / fluido / denso / congestionado) con los mismos algoritmos.
 - c. Dado el volumen limitado de datos, se recomienda empezar con modelos simples (regresión lineal, árbol de decisión) como baseline antes de modelos más complejos.
 - d. Validación temporal estricta: los datos de test deben ser temporalmente posteriores a los de entrenamiento.
4. **Análisis de correlaciones (aunque el modelo no sea robusto):**
 - a. Si los datos son insuficientes para un modelo predictivo fiable, el análisis de correlación entre variables climáticas y nivel de tráfico sigue siendo valioso: ¿Se reduce el tráfico con la lluvia? ¿Cuánto? ¿La temperatura tiene correlación con el uso de vehículo?
 - b. Este análisis exploratorio tiene valor independientemente de la calidad del modelo final.

Fuentes de datos

- **Cámaras DGT** — Valencia: <https://www.dgt.es/conoce-el-estado-del-trafico/camaras-de-trafico/?pag=1&prov=46> — Imágenes JPEG accesibles mediante URLs directas actualizadas periódicamente.
- **Estado del tráfico en tiempo real** — Open Data Valencia: <https://valencia.opendatasoft.com/explore/dataset/estat-transit-temps-real-estadotrafico-tiempo-real/> — Puede usarse como ground truth para validar los conteos de YOLOv.

- **Geoportal de tráfico Valencia:** <https://geoportal.valencia.es/portal/apps/webappviewer/index.html?id=3a302533ea51497f9a2af6b723cc9495> — Referencia geográfica de cámaras.
- **AEMET Open Data:** <https://opendata.aemet.es> — Variables meteorológicas en tiempo real para Valencia (requiere API key gratuita).
- **YOLOv8/v9 (Ultralytics):** <https://github.com/ultralytics/ultralytics> — pip install ultralytics — Clases relevantes COCO: car (2), motorcycle (3), bus (5), truck (7).

Más información de las fuentes en el anexo de recursos.

Aspectos MLOps y calidad a incorporar

- **Monitorización del pipeline de ingesta:** registrar en InfluxDB el número de cámaras activas en cada ciclo de captura, el tiempo de respuesta de cada URL y los errores de descarga. Un dashboard de Grafana debe mostrar el estado del pipeline en tiempo real.
- **Calidad de las imágenes:** implementar un filtro básico que descarte imágenes con baja calidad (imagen negra/oscura fuera de horario, imagen de baja varianza que puede indicar fallo de cámara).
- **Versionado en lakeFS:** etiquetar el dataset acumulado al final de cada semana del proyecto para poder reanalizar con modelos mejorados.
- **Registro en MLflow:** si se llega a entrenar el modelo predictivo de congestión, registrar todos los experimentos con los features utilizados, las métricas de evaluación y el análisis explícito de las limitaciones debidas al tamaño del dataset.

CASO 10: (J) Big Data: Benchmark Spark vs. Pandas (transversal, obligatorio)

Responsabilidad

Este caso de uso es **transversal y obligatorio** para el proyecto, pero **no debe realizarlo necesariamente cada equipo**. Será desarrollado principalmente por el equipo que tenga perfil de administración de sistemas / DevOps, que además configurará y administrará el clúster Big Data del ITI. El resto de equipos deben conocer los resultados y ser capaces de defenderlos.

Objetivo

Demostrar **empíricamente y con datos** las ventajas del procesamiento distribuido con Apache Spark en el clúster del ITI frente al procesamiento convencional con pandas, usando el dataset BDG2 (más de 53 millones de registros, ~1.67 GB de datos eléctricos limpios).

Problema a resolver

¿Por qué necesitamos Big Data si pandas funciona bien para datasets pequeños? Este caso de uso responde esa pregunta de forma definitiva: ejecuta las mismas operaciones sobre BDG2 con pandas en local y con Spark en el clúster, mide tiempos de ejecución, consumo de memoria y límites de escalabilidad, y documenta las conclusiones de forma pedagógicamente clara.

Escenario base (MVP)

1. **Configurar el clúster Hadoop/Spark** en los servidores del ITI (2-3 nodos):
 - a. Instalar Hadoop HDFS para almacenamiento distribuido.
 - b. Instalar Apache Spark con YARN como gestor de recursos.
 - c. Ingestar el dataset BDG2 completo en HDFS.
 - d. Documentar el proceso de instalación en el runbook.
2. **Diseñar el benchmark comparativo:**
 - a. Definir un conjunto de operaciones equivalentes que se ejecutarán con pandas y con Spark:
 - i. **Operación 1 — Filtrado:** Seleccionar todos los registros de edificios de tipo "Education" (metadatos) y las lecturas de electricidad del año 2016.
 - ii. **Operación 2 — Agregación temporal:** Calcular el consumo medio horario por tipo de edificio en todos los sitios.
 - iii. **Operación 3 — Join:** Unir los datos de consumo con los metadatos de edificio y los datos meteorológicos del mismo sitio y timestamp.
 - iv. **Operación 4 — Detección de outliers:** Calcular la media y la desviación estándar de consumo por edificio y marcar como outlier cualquier valor que supere $\text{media} \pm 3\sigma$.
 - b. Ejecutar cada operación:
 - i. Con **pandas** en un nodo del clúster (sin distribución): medir tiempo de ejecución con `time.time()` y consumo de memoria con la librería `memory_profiler`.

- ii. Con **PySpark** en el clúster (datos en HDFS): medir tiempo de ejecución (Spark UI) y observar la distribución de la carga entre nodos.
3. **Documentar los resultados** en un notebook comparativo con tablas, gráficos de barras (tiempo pandas vs. Spark por operación) y conclusiones claras:
 - a. ¿En qué punto pandas se queda sin memoria?
 - b. ¿Qué factor de aceleración ofrece Spark para operaciones sobre el dataset completo?
 - c. ¿Cuándo NO merece la pena usar Spark (datasets pequeños, latencia de arranque)?

Escenario avanzado (mejora nota)

- Añadir **Apache Kafka** para demostrar procesamiento de streaming: publicar datos BDG2 en un topic Kafka a alta velocidad y consumirlos con Spark Structured Streaming.
- Comparar el rendimiento del clúster con **2 nodos vs. 3 nodos** para demostrar el escalado horizontal.

CASO 11: (K) Edge Computing con Jetson Orin Nano (opcional)

Objetivo

Desplegar un **modelo de inferencia** (ocupación o calidad del aire) en el dispositivo NVIDIA Jetson Orin Nano, simulando el edge gateway de CENTINELA+, y demostrar el pipeline edge-to-cloud completo: Jetson → MQTT → InfluxDB → Grafana.

Caso de uso reservado para equipos avanzados con disponibilidad del dispositivo.

Escenario

En CENTINELA+, los dispositivos edge procesarán datos localmente antes de enviarlos al servidor central, reduciendo latencia y dependencia de conectividad. Este caso de uso demuestra esa capacidad de forma real.

Pipeline edge-to-cloud:

```
[Jetson Orin Nano]
  ↓ Leer datos de sensor (simulado con CSV)
  ↓ Inferencia local del modelo de ocupación (TensorRT / ONNX)
  ↓ Publicar resultado + datos crudos en MQTT (Mosquitto)
    ↓
[Servidor ITI - InfluxDB]
    ↓
[Grafana - Dashboard]
```

Modelo recomendado: El clasificador de ocupación del Caso D (Random Forest o red neuronal ligera), exportado a ONNX y optimizado con TensorRT para la GPU integrada del Jetson Orin Nano.